

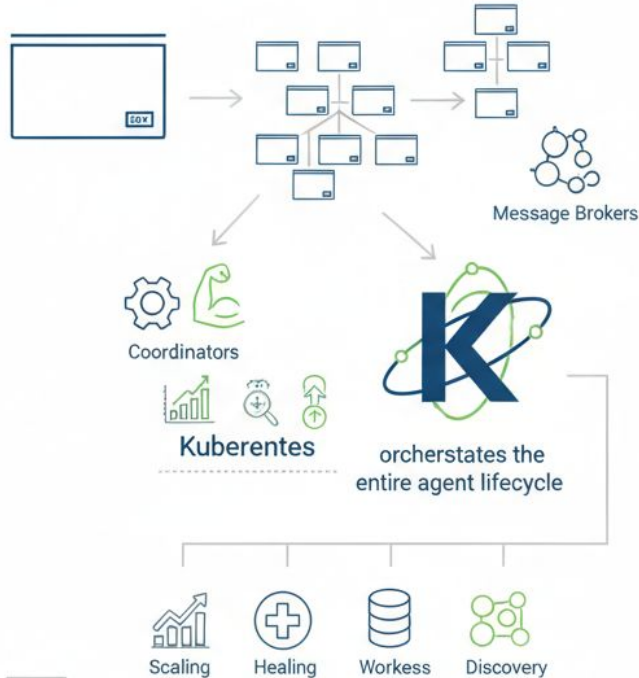


U.S. General Services  
Administration

# Chapter 4.3: Container Orchestration

US General Service Administration  
AI Community of Practice - March 2026

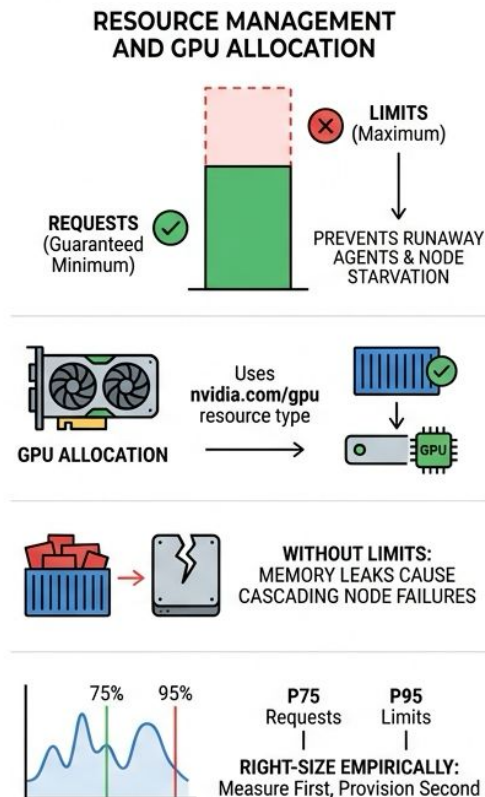
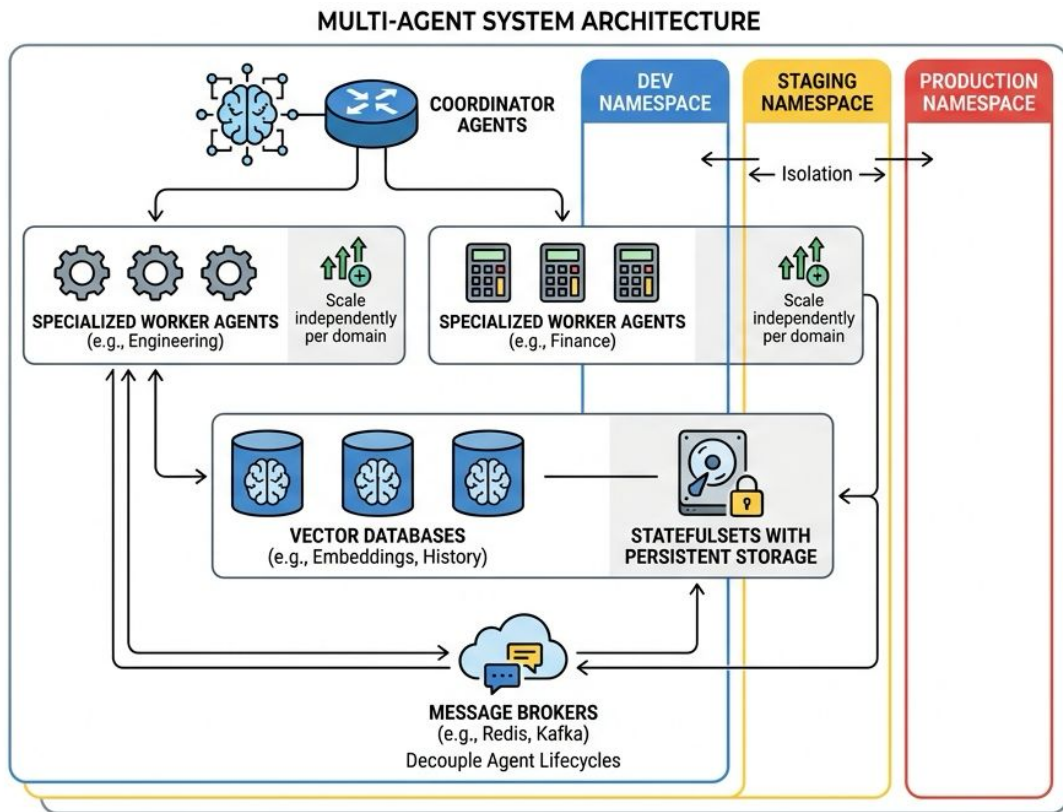
# Declarative Deployments and Desired State



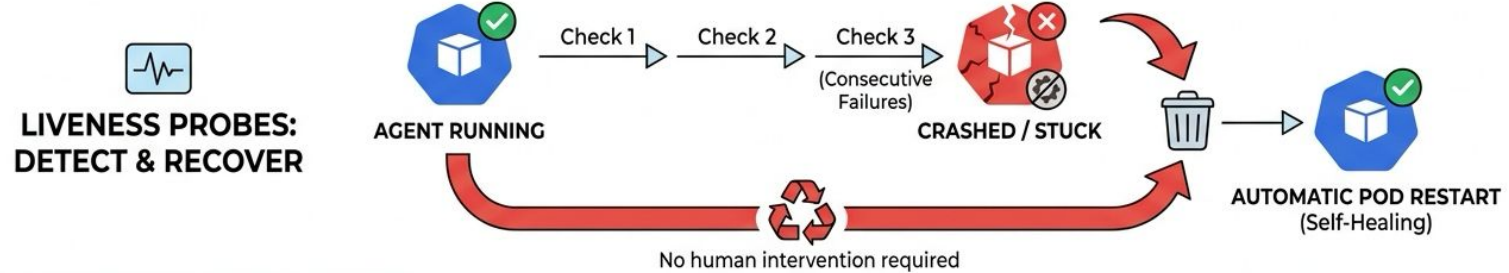
4.1



# Multi-Agent System Architecture in Kubernetes



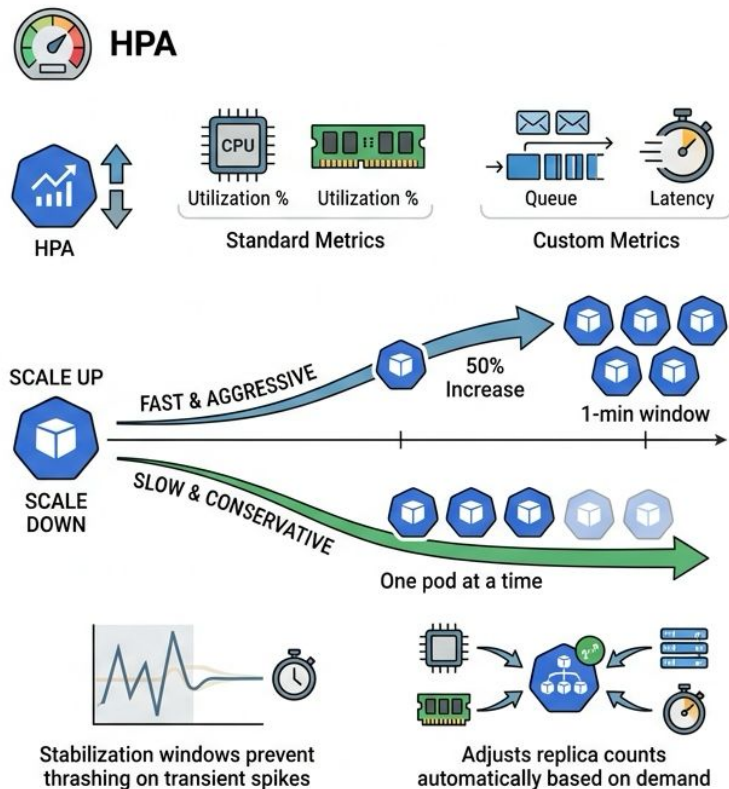
# Health Probes and Self-Healing



## CONTINUOUS SELF-HEALING SYSTEM

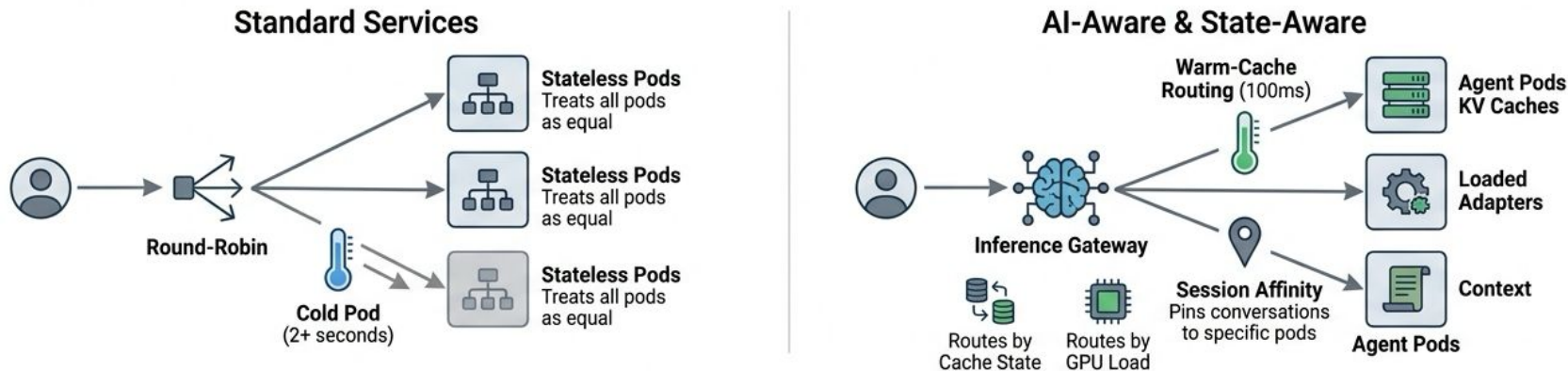
Kubernetes replaces failed pods and manages traffic flow automatically for resilient operations.

# Horizontal Pod Autoscaler and Scaling Policies



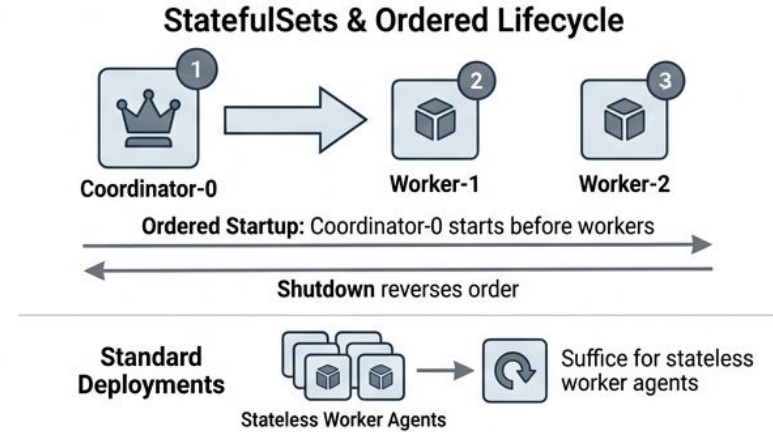
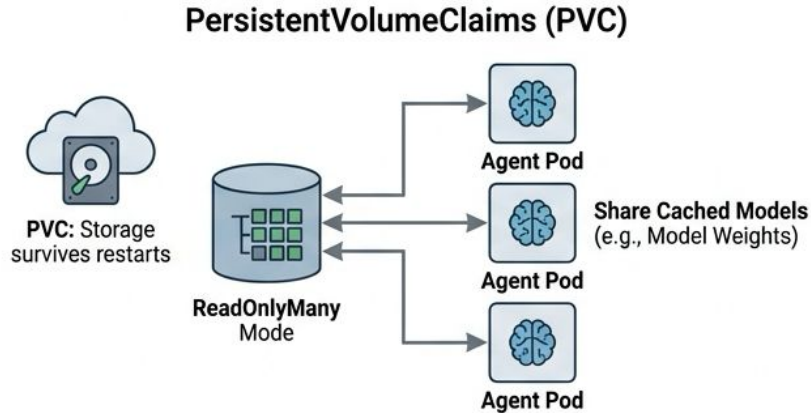
- HPA monitors metrics and adjusts replica counts automatically
- Standard metrics: CPU utilization and memory usage
- Custom metrics: queue depth, request latency
- Scale up fast and aggressively; scale down slowly
- Stabilization windows prevent thrashing on transient spikes

# AI-Aware Load Balancing



- Standard Services use round-robin, treating all pods as equal
- Agent pods carry state: KV caches, loaded adapters, context
- Warm-cache routing delivers 100ms; cold pod takes 2+ seconds
- Inference gateways route by cache state and GPU load
- Session affinity pins conversations to specific pods

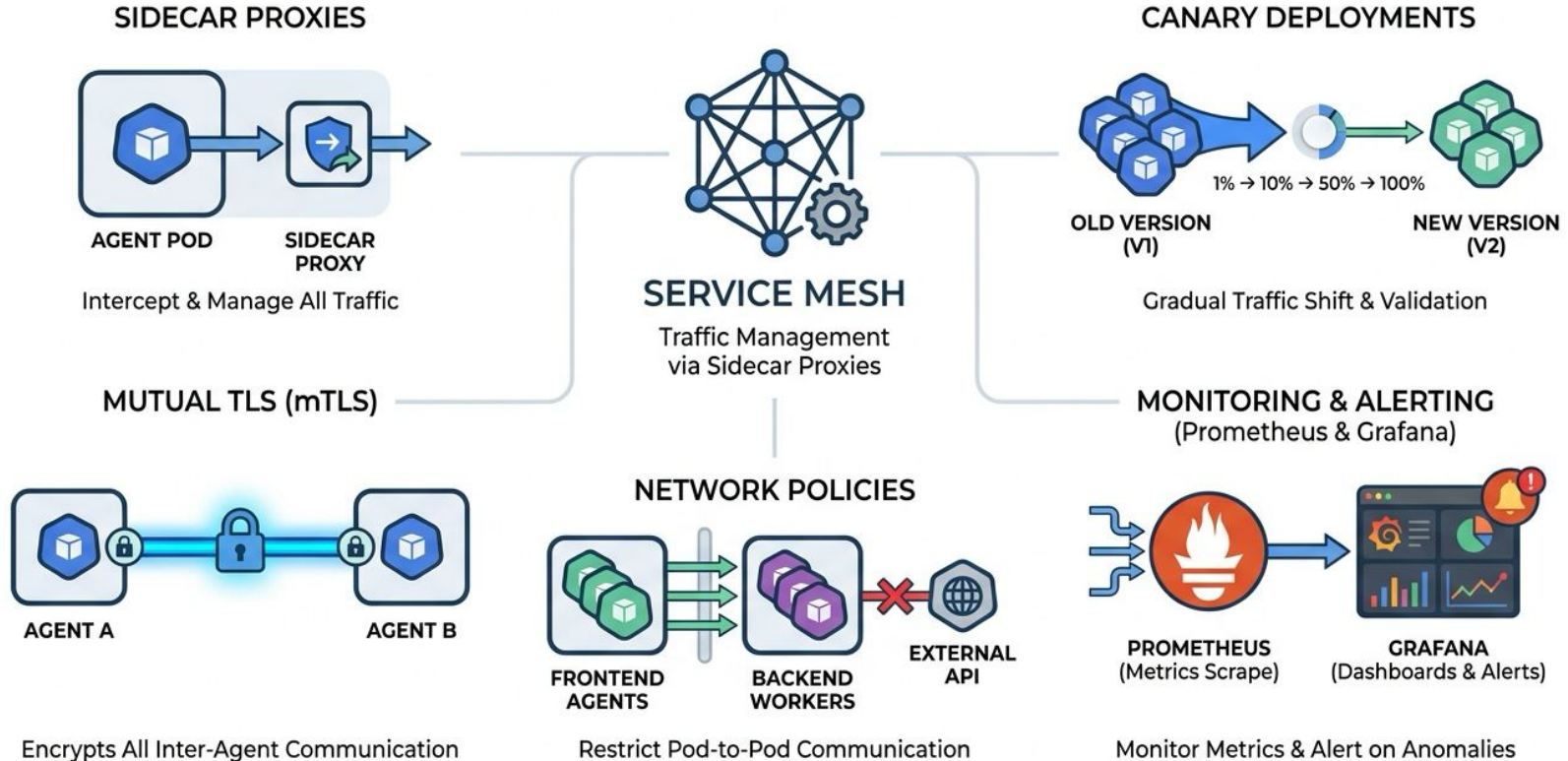
# Persistent Storage and StatefulSets



- PersistentVolumeClaims survive pod restarts and reschedules
- ReadOnlyMany mode lets all pods share cached models
- StatefulSets provide stable identity and ordered startup
- Coordinator-0 starts before workers; shutdown reverses order
- Standard Deployments suffice for stateless worker agents



# Service Mesh and Production Observability





# Common Pitfalls in Production



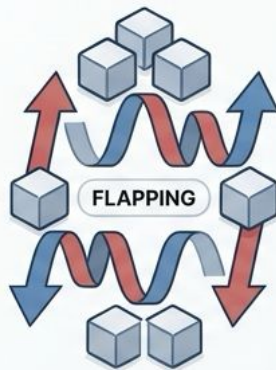
**No Resource Limits:**  
Cascading  
Out-of-Memory  
Failures

Uncontrolled consumption  
crashes nodes.



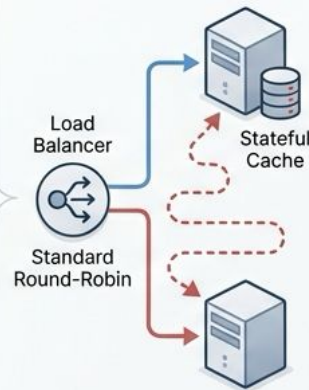
**Over-provisioning:**  
Wastes Budget

Low utilization at  
high cost.



**Aggressive  
Scale-Down:**  
Causes Flapping  
& Instability

Constant scaling creates  
system load.



**Standard Load  
Balancing:** Ignores  
Inference State  
Locality

Inefficient routing for  
stateful workloads.



**Premature  
Complexity:** Before  
Product-Market Fit

Over-engineering  
before validation.

# Worked Example -- Customer Service Agent System

---

- Coordinator: 2 replicas, 0.5 CPU, 1GB RAM, routing logic
  - Technical worker: 3 replicas, GPU-equipped, 8GB RAM
  - HPA scales workers from 3 to 20 based on queue depth
  - PVC shares 100GB model cache across all worker pods
  - LoadBalancer Service exposes coordinator to external traffic
- 

# Key Takeaways

---

- Kubernetes transforms manual ops into declarative desired state
  - Resource requests, limits, and probes are non-negotiable
  - Asymmetric scaling: fast up, slow down prevents flapping
  - Inference workloads need cache-aware routing, not round-robin
  - Next: Chapter 4.4 tackles performance profiling and optimization
- 